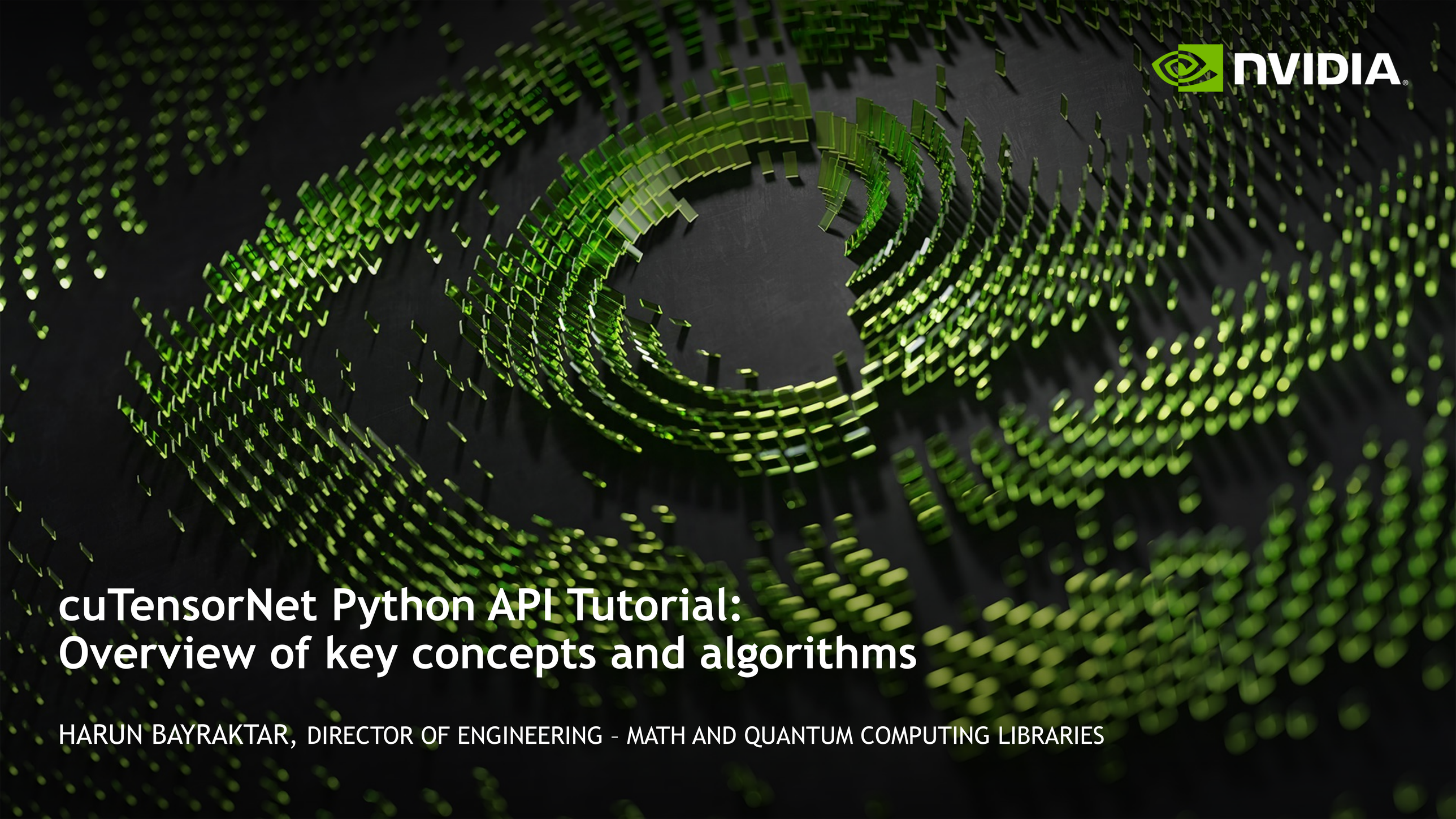
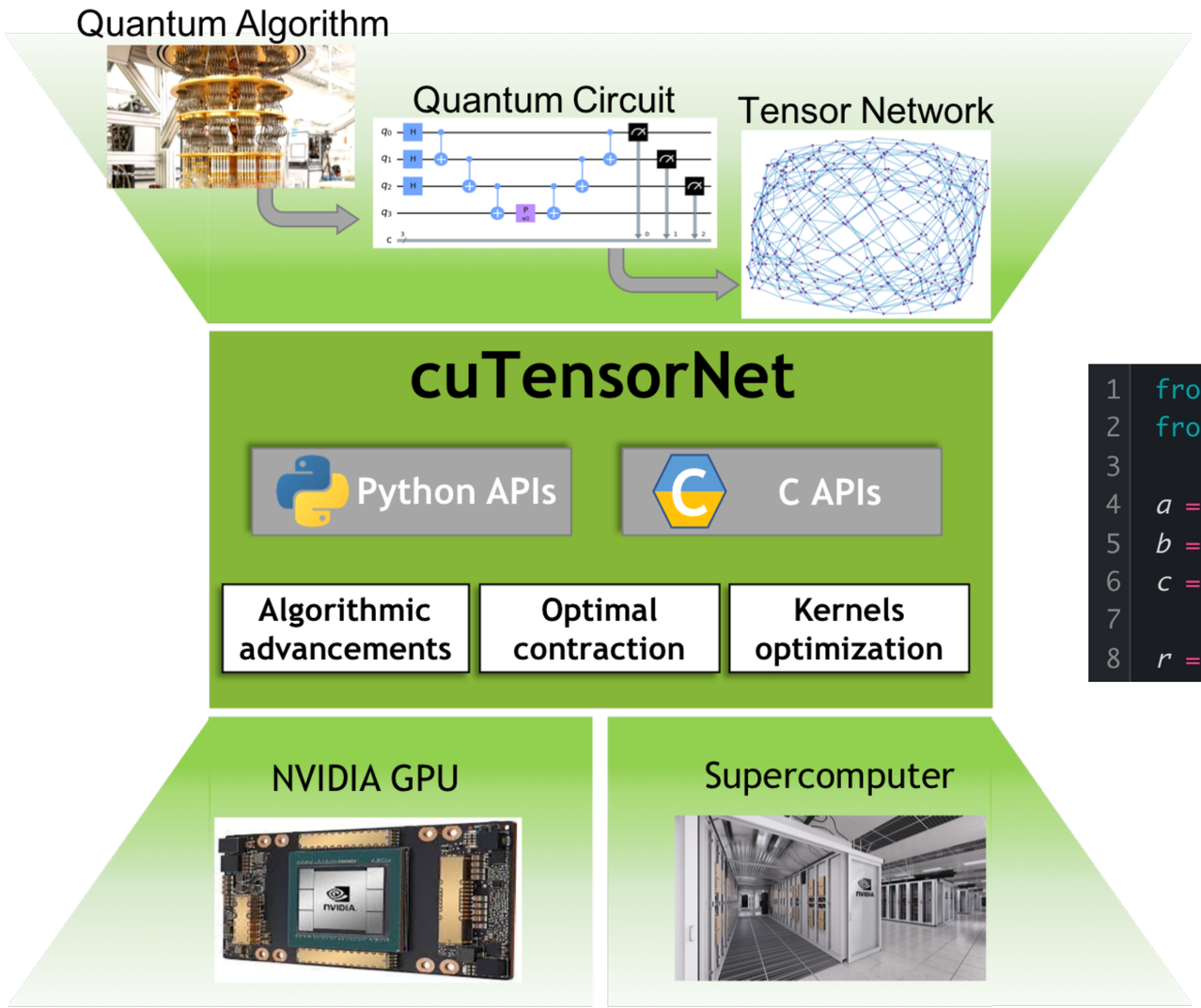


cuTensorNet Python API Tutorial: Overview of key concepts and algorithms

HARUN BAYRAKTAR, DIRECTOR OF ENGINEERING – MATH AND QUANTUM COMPUTING LIBRARIES

cuTensorNet

A LIBRARY TO ACCELERATE TENSOR NETWORK BASED QUANTUM CIRCUIT SIMULATION



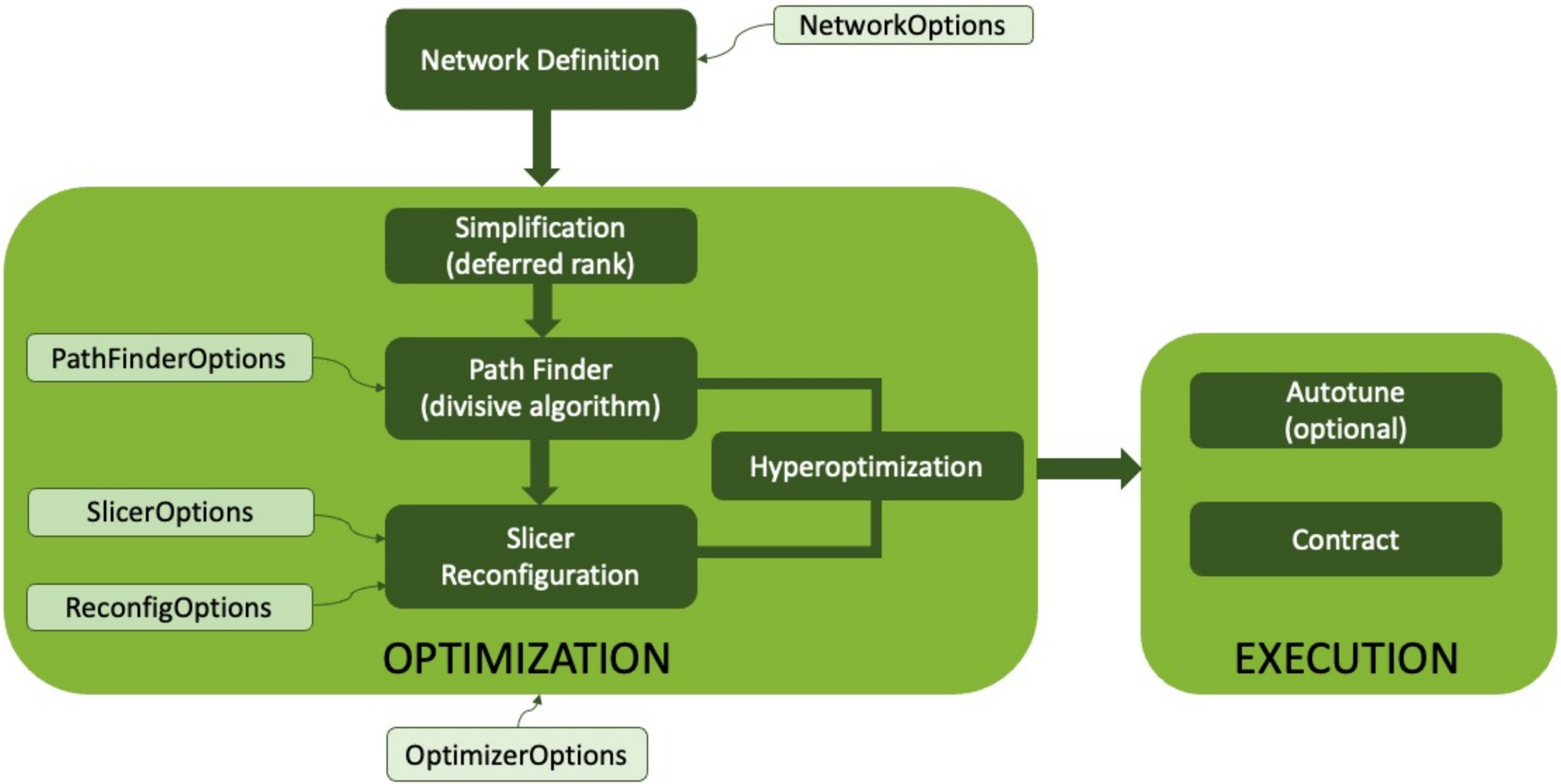

```python
from cuquantum import contract
from numpy.random import rand

a = (rand(96,64,64,96) - 0.5) * 100
b = (rand(96,64,64) - 0.5) * 100
c = (rand(64,96,64) - 0.5) * 100

r = contract("mhkn,ukh,xuy->mxny", a, b, c, optimize = {'samples' : 100})
```

https://developer.nvidia.com/blog/scaling-quantum-circuit-simulation-with-cutensornet/

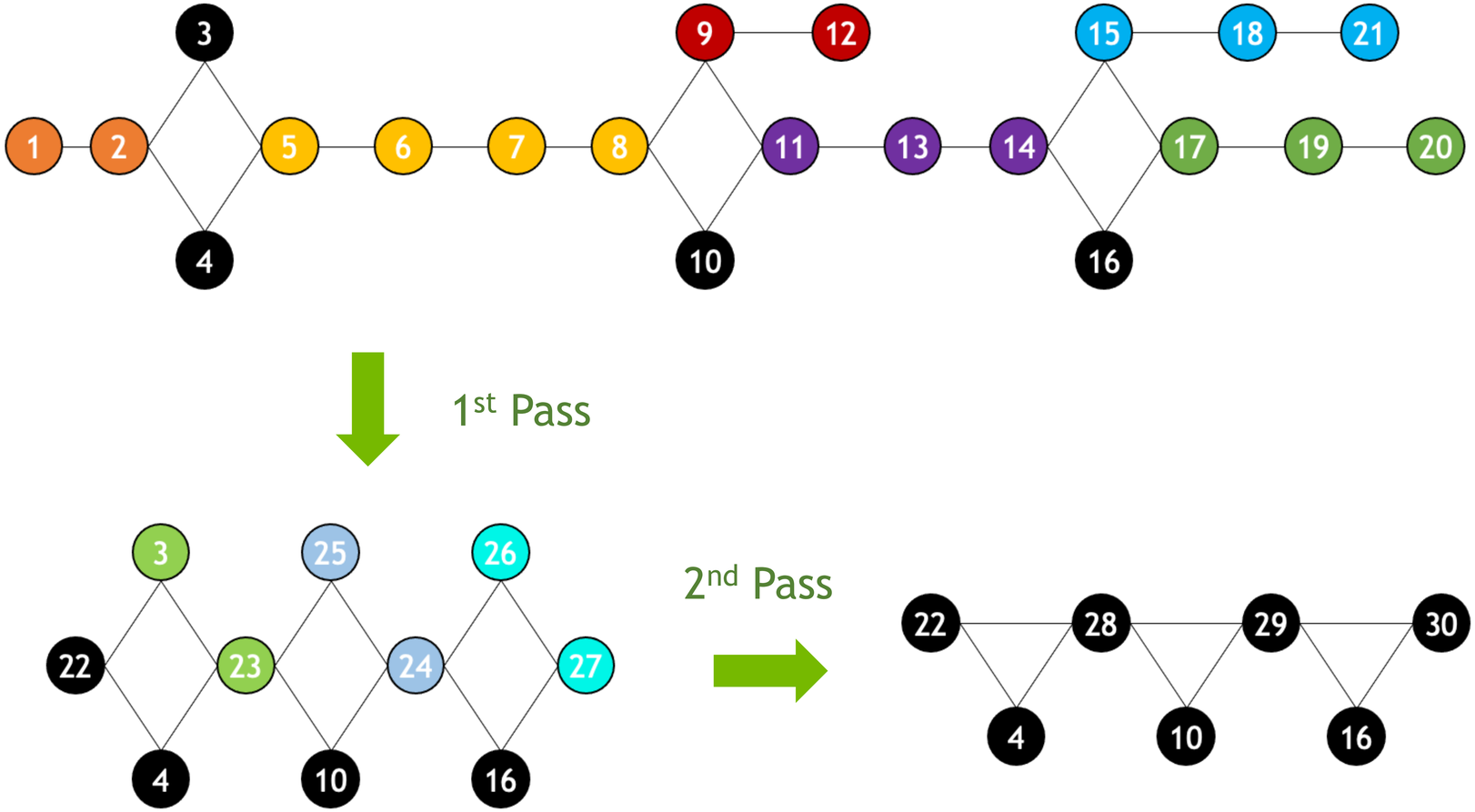
cuTensorNet Optimization & Flowchart



Tensor Network Simplification

Simplification aims to reduce the computational cost of contracting the tensor network through preprocessing.

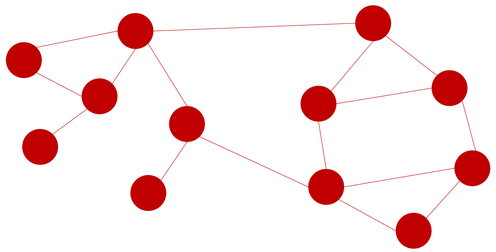
cuTensorNet implements *deferred rank-simplification*, which identifies those pairwise contractions that do not increase the rank (number of dimensions) of the resulting tensor and sequences them to be performed first as a path prefix. This essentially creates a smaller network for the divisive algorithm as well as for reconfiguration to process.



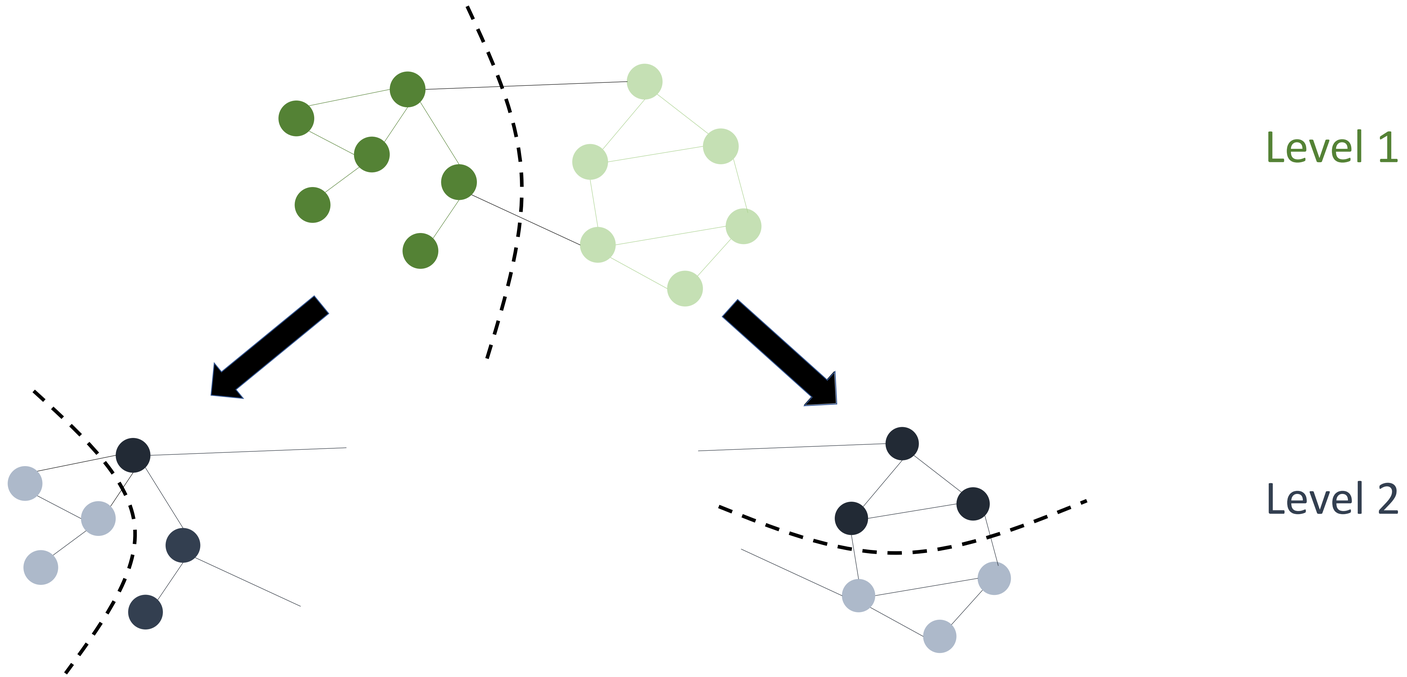
NVIDIA.

cuTensorNet Path Finder (Divisive Algorithm)

The tensor network is represented as a graph, with tensors as the vertices and modes that are contracted as the edges.



The graph is partitioned into the specified number of partitions (2 shown) recursively until the size of each partition is less than or equal to the specified cutoff size (3 shown). Exhaustive search or an agglomerative algorithm is used to find the contraction order within as well as between partitions, from which the contraction order for the complete tensor network is built.



The colors map to the partitioning level, and the shades at each level distinguish different partitions.

Tensor Network Slicing for Parallelism & Minimizing Memory Requirements

Slicing is a technique to select a subset of edges from a tensor network (corresponding to mode labels) for explicit summation.

A sliced network:

1. results in lower memory requirements (often with some computational overhead), and
2. allows for parallel execution.

cuTensorNet implements *dynamic slicing*, which interleaves slicing with reconfiguration.

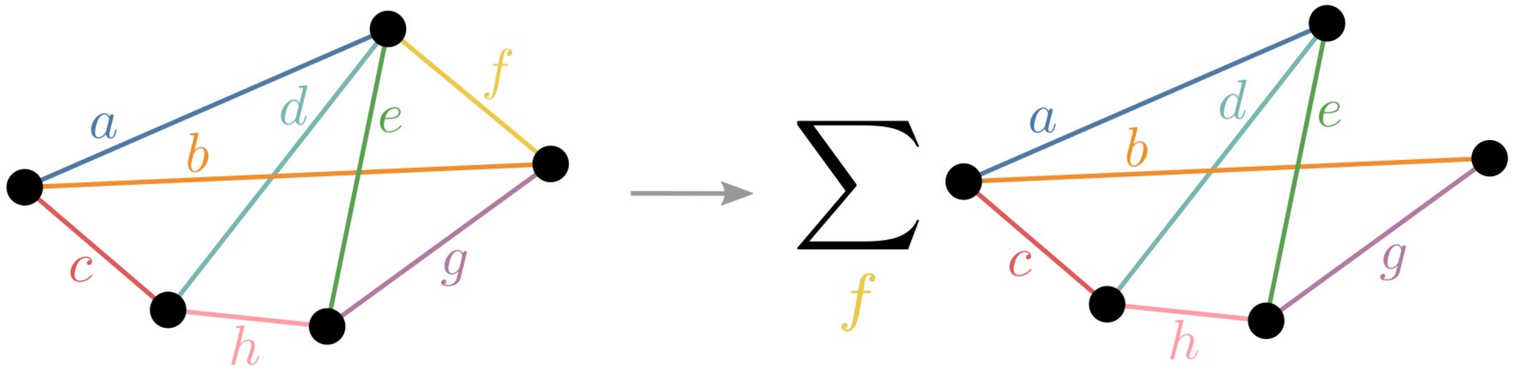


Image from https://github.com/jcmgray/cotengra by Johnnie Gray

Tensor Network Reconfiguration

The divisive algorithm computes a contraction path, which is a linearization of the contraction tree. The basic idea behind *reconfiguration* is to reduce the total contraction cost by reducing the contraction cost of portions (subtrees) of the contraction tree. The *number of leaves* in the subtree is typically chosen to be small enough so that the optimal algorithm can be used, and multiple *iterations* of reconfiguration are performed on different subtrees.

As mentioned earlier, if slicing is active cuTensorNet interleaves reconfiguration with slicing to keep the contraction cost low.

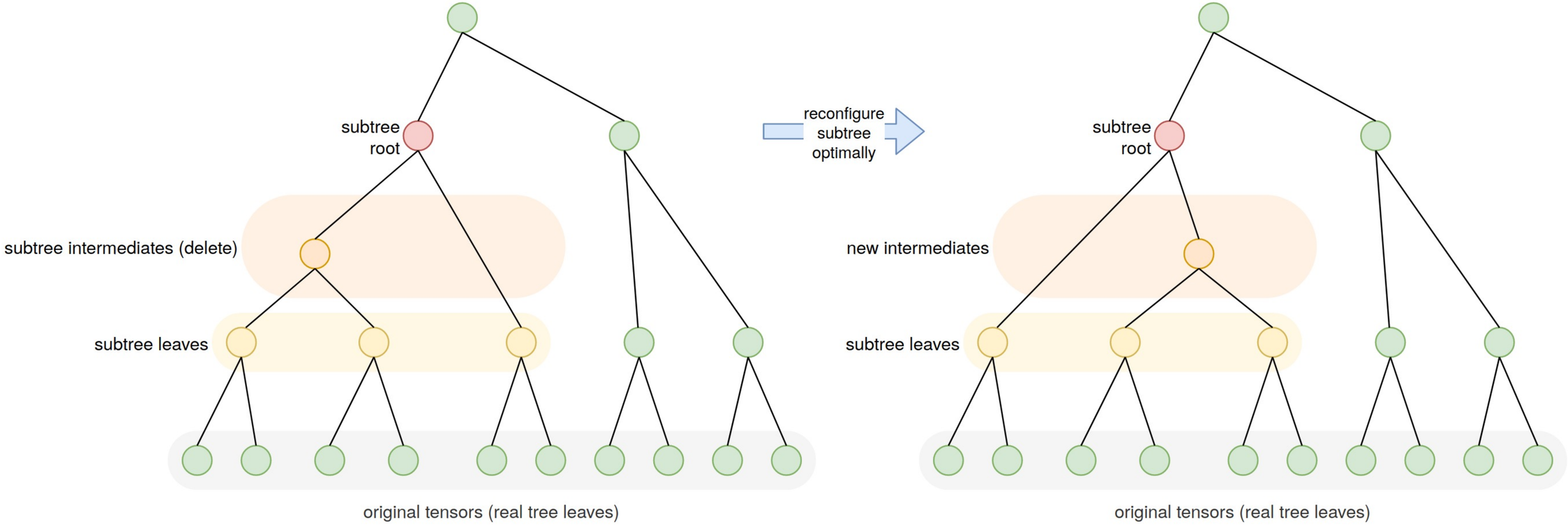
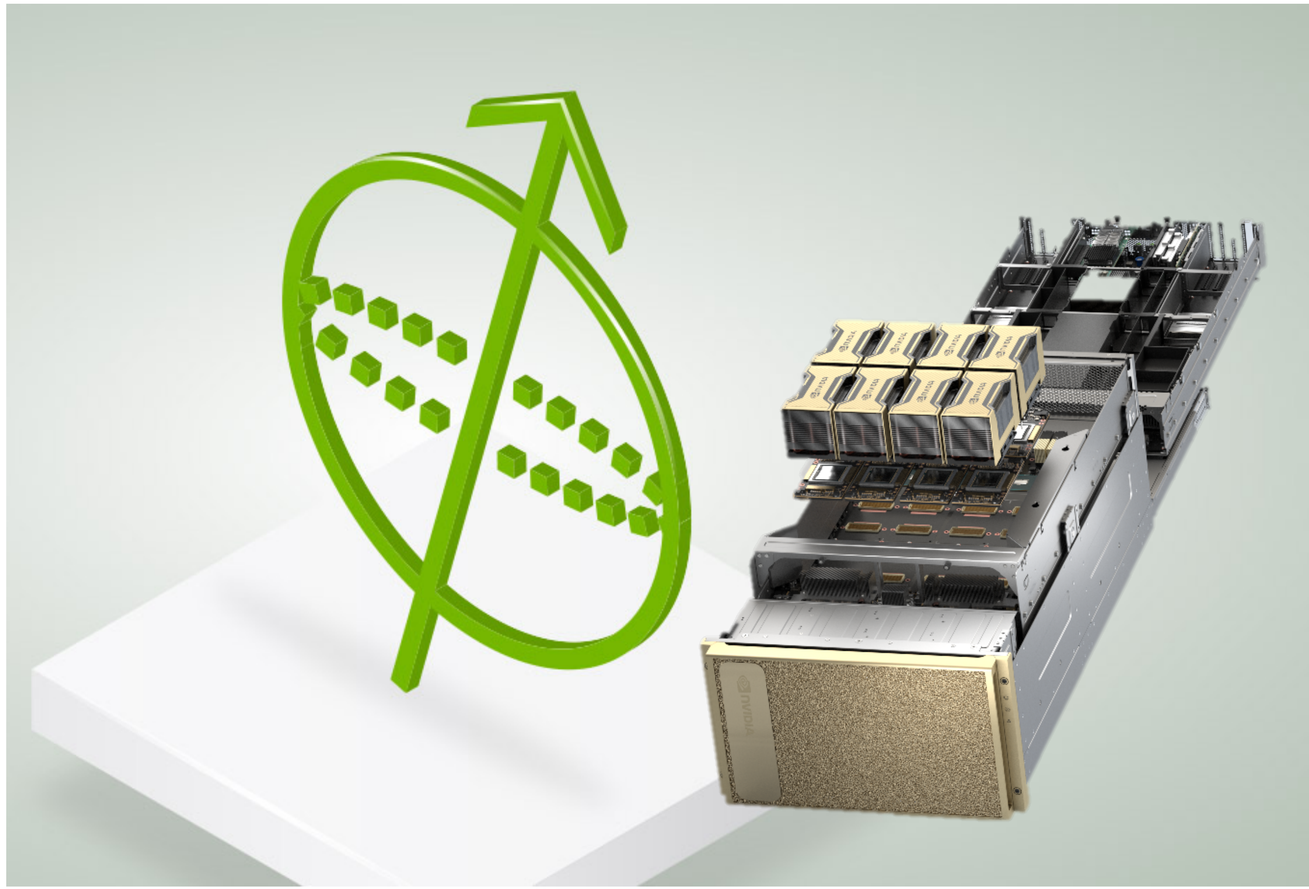


Image from https://github.com/jcmgray/cotengra by Johnnie Gray

NVIDIA.

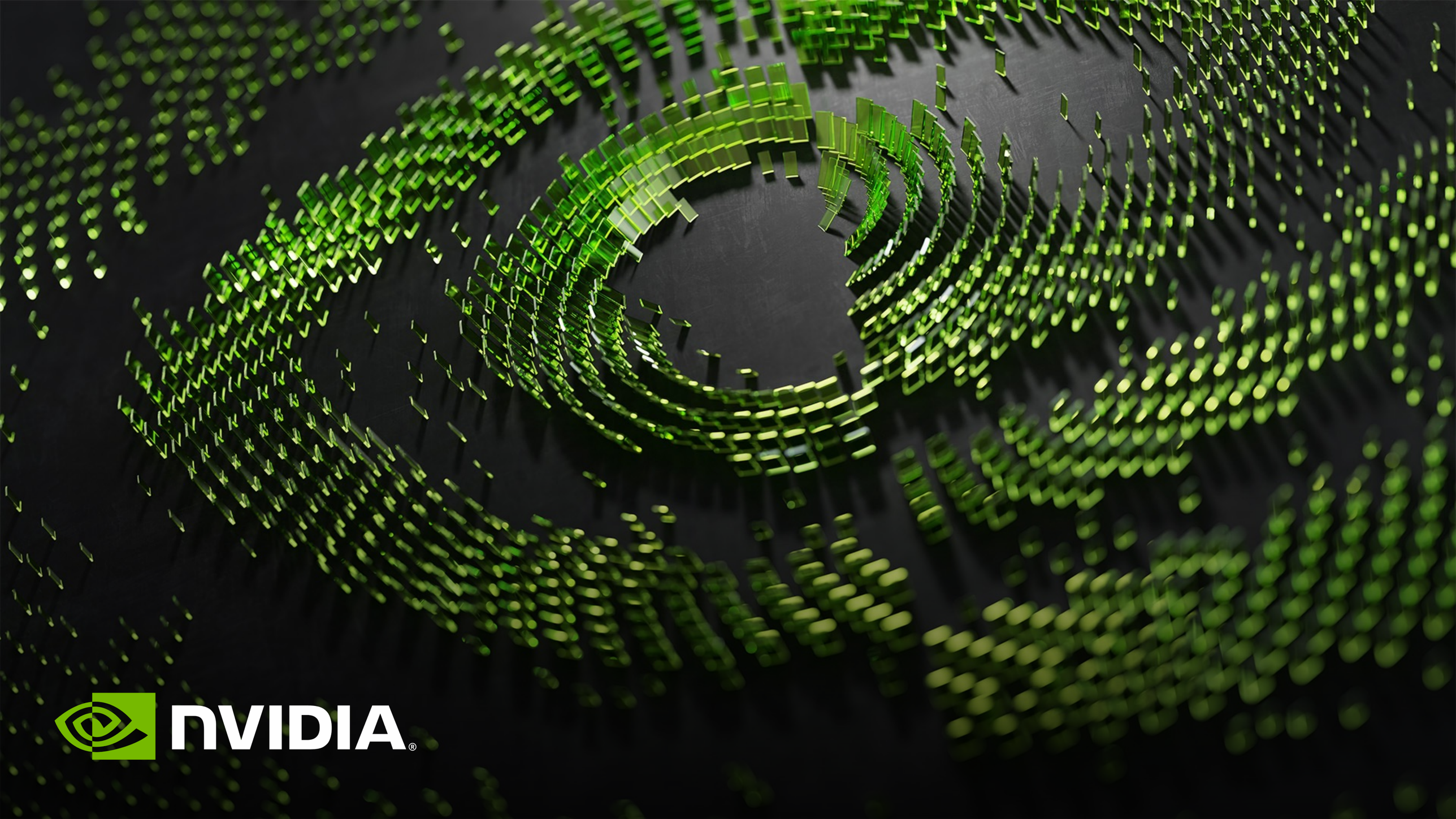
Where can I find out more about cuQuantum?

- cuQuantum SDK is available for download at:
  - https://developer.nvidia.com/cuquantum-sdk
- The cuQuantum Appliance container is available from NGC:
  - https://catalog.ngc.nvidia.com/orgs/nvidia/containers/cuquantum-appliance
  - Cirq/Qsim Quantum Circuit Simulator accelerated with cuStateVec and optimized for multi-GPU execution on the DGX A100
  - A container with a tensor network based simulator is coming later this year
- cuTensorNet technical blog
  - https://developer.nvidia.com/blog/scaling-quantum-circuit-simulation-with-cutensornet/
- cuStateVec technical blog
  - https://developer.nvidia.com/blog/accelerating-quantum-circuit-simulation-with-nvidia-cuStateVec/
- *Please provide feedback when you get a chance to try it out*



To learn more about tensor network contraction optimization see:
Johnnie Gray and Stefanos Kourtis, "Hyper-optimized tensor network contraction," Quantum, volume 5, 2021.

NVIDIA.


NVIDIA